

DOKUMENTASI TUGAS PROYEK DATA ENGINEERING

TMDB Movie Pipeline

End-to-End Data Pipeline: Bronze → Silver → Gold → Dashboard

Tools & Teknologi

PostgreSQL · dbt · PySpark · Apache NiFi · Apache Kafka · Debezium

ClickHouse · MinIO · Streamlit · Docker · Python

By : Nabila Hulwana Z.

Mini Bootcamp Data Engineering rubythalib.ai | Instruktur: Kak Guntur Periode
Periode : Februari – April 2026

Link projek di github : <https://github.com/nabilahlw/tmdb-pipeline-project.git>

Gambaran Umum Proyek

Proyek ini membangun pipeline data end-to-end untuk dataset film TMDb (The Movie Database) menggunakan arsitektur Medallion (Bronze → Silver → Gold). Data bersumber dari dua tempat: dataset Kaggle (4.803 film) dan TMDb API live.

Sumber Data:

TMDb API Live — <https://www.themoviedb.org/subscribe/developer>

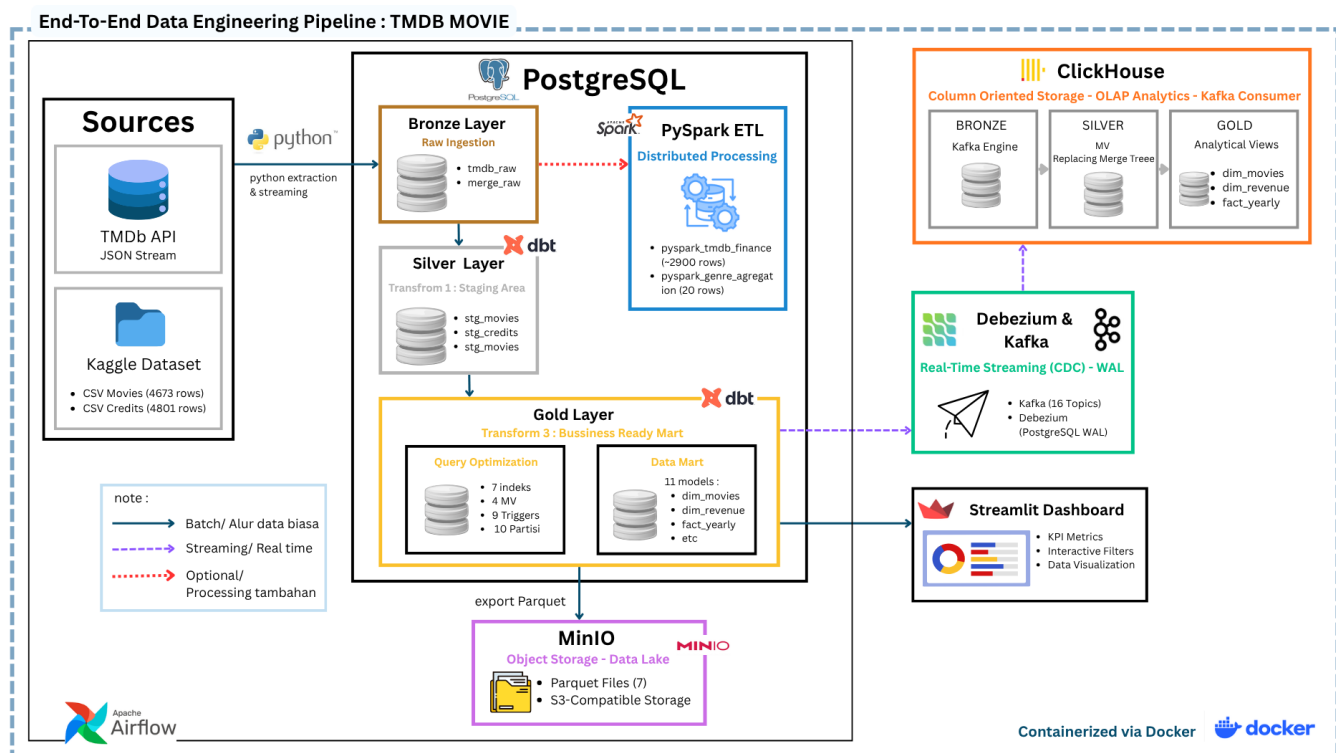
Kaggle Dataset — <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>

Ringkasan Pipeline

Step	Komponen	Output
Step 1	Bronze Layer — Ingestion	tmdb_raw (80 rows), merge_raw (4.803 rows)
Step 2	Silver/Gold — dbt Transformation	11 mart model di schema silver_mart
Step 3	Query Optimization	4 MV, 10 partisi, 7 index, 9 triggers
Step 4	PySpark ETL	pyspark_tmdb_finance, pyspark_genre_aggregation
Step 5	Apache NiFi Visual Pipeline	nifi_tmdb_movie
Step 6	Kafka + Debezium CDC	16 topics, streaming real-time
Step 7	ClickHouse OLAP	9 objek, 9.604 rows dim_movies
Step 8	MinIO Object Storage	7 file Parquet, 2.7 MiB
Step 9	Streamlit Dashboard	Dashboard interaktif dengan filter
Step 10	Apache Airflow / DAG Orchestration	Otomasi 4 step (API, CSV ke PostgreSQL dan MinIO)

Arsitektur Pipeline

Pipeline menggunakan pola EtLT (Extract → Load → Transform → Load → Transform) modern, di mana data dikumpulkan dulu dalam bentuk raw lalu ditransformasi secara bertahap mengikuti Medallion Architecture.



Sebelum menjalankan proyek ini semua container docker yang terlibat harus terbuka.

	Name	Container ID	Image	Port(s)	CPU	Actions
☐	tmdb-pipeline-project	-	-	-	N	
☐	tmdb_airflow_scheduler	33c49eaa546f	apache/airflow:2.8.1		N	
☐	tmdb_airflow_web	457fb22a156e	apache/airflow:2.8.1	8082:8080	N	
☐	tmdb_airflow_init	de3683fc0b9a	apache/airflow:2.8.1		N	
☐	tmdb_debezium	a8c8d14f0c42	debezium/connect:2.4	8083:8083	N	
☐	tmdb_pgadmin	c0e92ce2849b	dpage/pgadmin4	5050:80	N	
☐	tmdb_kafka	17fcf1c8b125	confluentinc/cp-kafka:7.4.0	29092:29092	N	
☐	tmdb_kafka_man	8a0a6fc83902	hlebaltbau/kafka-manager:latest	9000:9000	N	
☐	tmdb_postgres	22827ae4eea3	postgres:13	5432:5432	N	
☐	tmdb_clickhouse	4ce4f4eac5e7	clickhouse/clickhouse-server:23.8	18123:8123 Show all ports (2)	N	
☐	tmdb_zookeeper	7dda08543494	confluentinc/cp-zookeeper:7.4.0		N	
☐	tmdb_minio	4a23ad4f533a	minio/minio:latest	9002:9000 Show all ports (2)	N	

Step 1 — Bronze Layer: Environment Setup & Data Ingestion

Bronze layer adalah lapisan pertama dalam Medallion Architecture yang menyimpan data raw tanpa transformasi apapun. Sumber data berasal dari dua tempat yang kemudian digabungkan.

1.1 Infrastruktur Docker (Container)

Container	Image	Port	Fungsi
tmdb_postgres	postgres:13	5432:5432	Database utama PostgreSQL
tmdb_pgadmin	dpage/pgadmin4	5050:80	GUI monitoring database

Konfigurasi koneksi pgAdmin: Host = postgres (nama container, BUKAN localhost) | Port = 5432 | Username = admin | Password = adminadmin | Database = tmdb_db

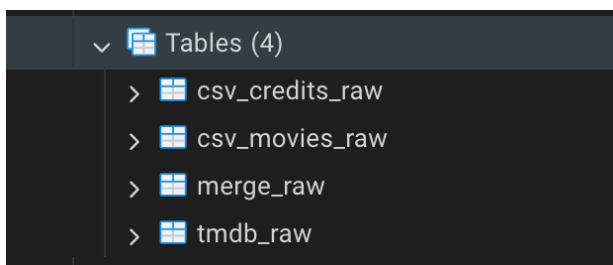
1.2 Sumber Data

Sumber	Format	Jumlah Data	Keterangan
TMDb API live (tmdb_raw)	JSON	80 rows	Endpoint /movie/popular, authentication via API key
Kaggle CSV — tmdb_5000_movies	CSV	4.673 rows	Data historis film dengan kolom budget, revenue, genres
Kaggle CSV — tmdb_5000_credits	CSV	4.801 rows	Data cast dan crew per film

1.3 Struktur Bronze di PgAdmin

Masuk ke <http://localhost:5050/browser/>

Nama proyek : tmdb_postgres, lalu tmdb_db → Schemas → public → Tables → merge_raw, tmdb_raw, csv_movies_raw



1.4 Struktur Tabel tmdb_raw (15 Kolom) dari API Tmdb

Kolom	Tipe Data	Penjelasan
-------	-----------	------------

id	bigint	ID unik film dari TMDb
title	text	Judul film dalam bahasa Inggris
original_title	text	Judul asli film
original_language	text	Kode bahasa (en, fr, ko, dll.)
overview	text	Deskripsi / sinopsis film
genre_ids	text	List ID genre (masih bentuk string JSON)
release_date	text	Tanggal rilis (format: YYYY-MM-DD)
popularity	double precision	Skor popularitas dari TMDb
vote_average	double precision	Rating rata-rata (skala 0-10)
vote_count	bigint	Jumlah vote yang masuk
adult	boolean	Konten dewasa (True/False)
video	boolean	Apakah berupa video (biasanya False)
backdrop_path	text	Path gambar backdrop
poster_path	text	Path poster film
category	text	Kategori (popular/top_rated/dll.)

1.4 Struktur Tabel merge_raw (22 Kolom, 4.803 rows)

Dua file CSV digabungkan menjadi tabel merge_raw dengan shape (4803, 22). Kolom-kolom yang tersedia:

budget, genres, homepage, id, keywords, original_language, original_title, overview, popularity, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, vote_average, vote_count, cast, crew

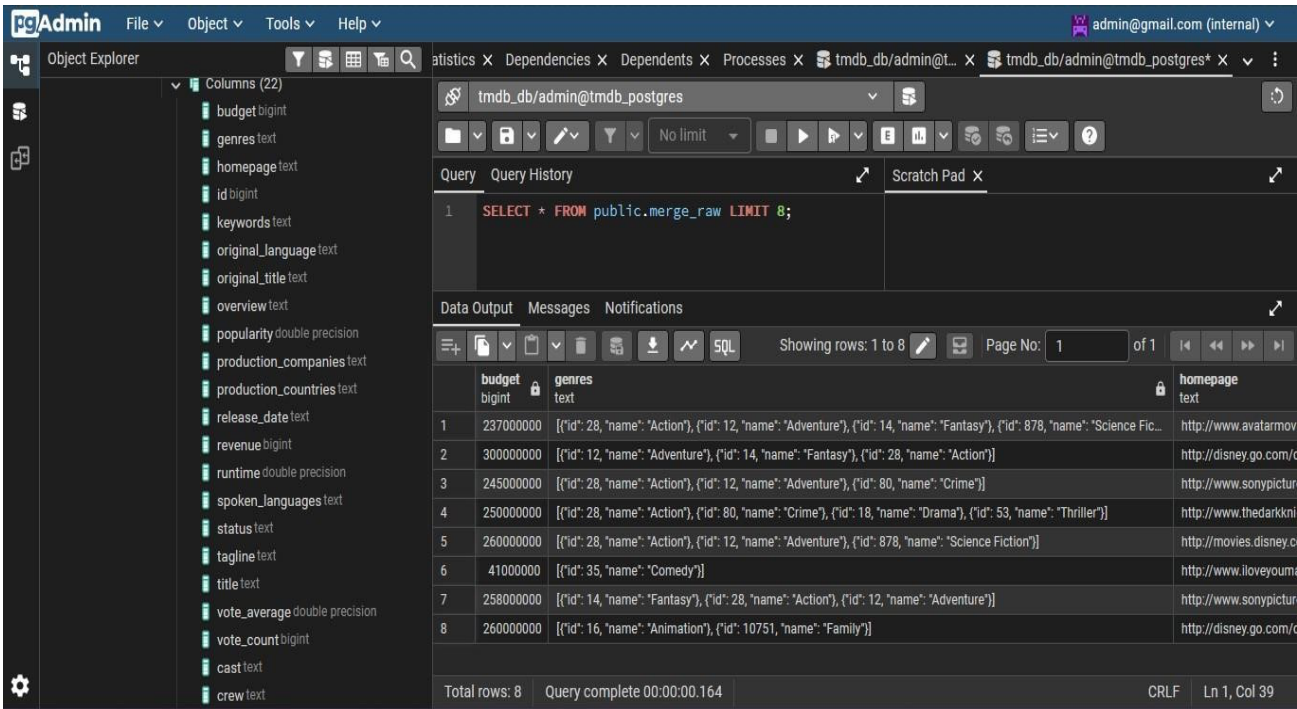
1.5 Kolom JSON yang Perlu Di-parse

NOTE: Kolom-kolom berikut masih dalam format STRING JSON dan akan di-parse di step selanjutnya.

Kolom JSON	Contoh Isi	Di-parse di Step
genres	[{"id":28,"name":"Action"},...]	Step 2 (dbt) & Step 4 (PySpark)
cast	[{"name":"Sam Worthington",...}]	Step 2 (dbt) & Step 4 (PySpark)
crew	[{"job":"Director","name":"..."}]	Step 2 (dbt) & Step 4 (PySpark)
keywords	[{"id":1,"name":"avatar"},...]	Step 2 (dbt)
production_companies	[{"name":"Warner Bros"},...]	Step 2 (dbt) & Step 5 (NiFi)
production_countries	[{"iso":"US","name":"..."},...]	Step 2 (dbt)

spoken_languages	[{"iso": "en", "name": "English"}, ...]	Step 2 (dbt)
------------------	---	--------------

Validasi tabel merge_raw di Postgres



Step 2 — Silver & Gold Layer: dbt Transformation

dbt (data build tool) digunakan untuk mentransformasi data dari Bronze layer menjadi tabel-tabel analitik yang bersih dan terstruktur. Arsitektur menggunakan 2 layer: Staging (pembersihan) dan Mart (analitik).

Letak di PgAdmin = tmdb_db → Schemas → **silver_mart** → 11 Tables (dim_movies, dim_revenue, dim_genres, dll)
→ **Views** (stg_movies, stg_credits, stg_tmdbapi)

2.1 Environment dbt

Komponen	Nilai
dbt version	1.11.7 (latest)
Python version	3.11.9
PostgreSQL adapter	dbt-postgres 1.10.0 — OK
Target schema	silver_mart
Jumlah model	3 staging + 11 mart = 14 model total

2.2 Staging Layer (3 Model)

Model	Sumber	Transformasi Utama
stg_movies.sql	merge_raw	Handle 2 format tanggal (DD/MM/YYYY & YYYY-MM-DD), parse semua kolom JSON, normalisasi popularity & vote_average
stg_credits.sql	merge_raw	Ekstrak lead actor, top 3 cast, director, screenplay writer, producer, cinematographer, music composer dari JSON
stg_tmdbapi.sql	tmdb_raw	Fix format tanggal, handle genre_ids sebagai PostgreSQL native array

2.3 Mart Layer (11 Model) — Schema: silver_mart

Model	Deskripsi	Kolom Kunci
dim_movies	Tabel master film lengkap, gabungan 3 sumber	movie_id, title, primary_genre, director, lead_actor, release_year
dim_genres	Ringkasan performa per genre	genre, total_films, total_revenue, avg_rating, top_grossing_film
dim_directors	Profil karir sutradara	director, total_films, director_roi_pct, filmography
dim_actor	Profil karir aktor (lead + supporting)	actor_name, lead_rate_pct, frequent_director, best_performing_film
dim_country	Ringkasan per negara produksi	primary_country, total_films, most_common_genre, most_active_director
dim_language	Analisis per bahasa film	original_language, total_films, dominant_country, top_revenue_genre
dim_company	Profil rumah produksi	main_production_company, blockbuster_rate_pct, most_active_director
dim_revenue	Fakta keuangan per film	performance_label, roi_pct, popularity_tier, profit
fact_vs	Rekonsiliasi 2 sumber data (CSV vs API)	in_api, in_kaggle, popularity_diff, api_vote_average
fact_yearly	Tren industri per tahun per genre	release_year, primary_genre, blockbuster_count, loss_count
dim_yearlysum	Ringkasan industri per tahun (1 baris/tahun)	most_active_director, top_grossing_film, most_common_genre

2.4 Kategori Performa Film (dari mart layer dim_revenue)

Label	Kriteria	Keterangan
Mega Blockbuster	$\text{revenue} \geq \text{budget} \times 3$	Film sukses besar, balik modal 3x lipat
Blockbuster	$\text{revenue} \geq \text{budget} \times 2$	Film sukses, balik modal 2x lipat
Profitable	$\text{revenue} > \text{budget}$	Untung tapi tidak signifikan

Break Even	revenue = budget	Tidak untung tidak rugi
Loss	revenue < budget	Film merugi

Memastikan 11 tabel mart sudah ada di silver_mart plus 2 tabel bronze di public di PgAdmin
<https://github.com/nabilahlw/tmdb-pipeline-project.git>

	table_schema name	table_name
1	public	merge_raw
2	public	tmdb_raw
3	silver_mart	dim_actor
4	silver_mart	dim_company
5	silver_mart	dim_country
6	silver_mart	dim_directors
7	silver_mart	dim_genres
8	silver_mart	dim_language
9	silver_mart	dim_movies
10	silver_mart	dim_revenue
11	silver_mart	dim_yearlysu...
12	silver_mart	fact_vs
13	silver_mart	fact_yearly

Step 3 Query Optimization: EXPLAIN, Index, Partition, MV

Step 3 membuktikan kemampuan optimasi performa database menggunakan 4 teknik utama: EXPLAIN untuk profiling, Index untuk mempercepat lookup, Partition untuk membagi data besar, dan Materialized View sebagai cache agregasi.

Letak Optimization di PgAdmin : tmdb_db → Schemas → silver_mart → silver_mart → Tables → table dim_movies → klik kanan → Properties → Indexes (lihat 7 index di sana)

3.1 Hasil Validasi Akhir

Objek	Jumlah	Detail
Indexes	7	idx_dim_movies_director, idx_dim_movies_genre, idx_dim_movies_year, idx_dim_movies_genre_rating, + 3 lainnya
Materialized Views	4	mv_genre_summary, mv_revenue_summary, mv_director_performance, mv_country_performance

Triggers	9	3 fungsi auto-refresh × 3 event (INSERT/UPDATE/DELETE) = 9 trigger
Partitions	10	fact_yearly_1920s hingga fact_yearly_2010s (per dekade)

3.2 EXPLAIN — Analisis Performa Query

EXPLAIN dijalankan pada dim_movies dan fact_yearly sebagai baseline sebelum optimasi. Output menunjukkan Seq Scan (membaca seluruh tabel) yang kemudian berubah menjadi Index Scan setelah index dibuat — cost turun drastis.

3.3 Partisi fact_yearly — Per Dekade

Tabel fact_yearly dibagi menjadi 10 sub-tabel berdasarkan release_year menggunakan PARTITION BY RANGE. Ketika query filter WHERE release_year = 2010, PostgreSQL hanya membaca partisi fact_yearly_2010s — bukan seluruh tabel.

	QUERY PLAN
	text
1	Sort (cost=13.24..13.31 rows=27 width=36)
2	Sort Key: release_year
3	-> HashAggregate (cost=12.26..12.60 rows=27 width=36)
4	Group Key: release_year
5	-> Seq Scan on fact_yearly (cost=0.00..10.45 rows=363 w...
6	Filter: ((release_year >= 1990) AND (release_year <= 20...

Lalu membuat indeks

Query	Query History
1	CREATE INDEX idx_dim_movies_director ON silver_mart.dim_movies (director);
2	CREATE INDEX idx_dim_movies_genre ON silver_mart.dim_movies (primary_genre);
3	CREATE INDEX idx_dim_movies_year ON silver_mart.dim_movies (release_year);
4	CREATE INDEX idx_dim_movies_genre_rating ON silver_mart.dim_movies (primary_genre, vote_average DESC);
5	CREATE INDEX idx_dim_revenue_genre ON silver_mart.dim_revenue (primary_genre);
6	CREATE INDEX idx_dim_revenue_year ON silver_mart.dim_revenue (release_year);
7	CREATE INDEX idx_dim_revenue_perf ON silver_mart.dim_revenue (performance_label);

Data Output	Messages	Notifications
CREATE INDEX		
Query returned successfully in 381 msec.		

Membuktikan indeks bekerja, COST turun

```

Query Query History
2 FROM silver_mart.dim_revenue WHERE director = 'Steven Spielberg';
3
4 EXPLAIN SELECT title, vote_average, popularity, revenue
5 FROM silver_mart.dim_revenue
6 WHERE primary_genre = 'Action' AND vote_average > 7.0
7 ORDER BY revenue DESC LIMIT 10;

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No:

QUERY PLAN
text

1	Limit (cost=79.52..79.53 rows=1 width=31)
2	-> Sort (cost=79.52..79.53 rows=1 width=31)
3	Sort Key: revenue DESC
4	-> Bitmap Heap Scan on dim_revenue (cost=8.69..79.51 rows=1 width=31)
5	Recheck Cond: (primary_genre = 'Action':text)
6	Filter: (vote_average > 7.0)
7	-> Bitmap Index Scan on idx_dim_revenue_genre (cost=0.00..8.69 rows=588 width=31)
8	Index Cond: (primary_genre = 'Action':text)

```

Query Query History
1 -- Tabel LAMA (tanpa partisi)
2 EXPLAIN SELECT * FROM silver_mart.fact_yearly WHERE release_year = 2010;
3
4 -- Tabel BARU (dengan partisi) - bandingkan cost-nya
5 EXPLAIN SELECT * FROM public.fact_yearly_partisi WHERE release_year = 2010;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

QUERY PLAN
text

1	Seq Scan on fact_yearly_2010s fact_yearly_partisi (cost=0.00..3.28 rows=15 width=31)
2	Filter: (release_year = 2010)

3.4 Materialized View + Auto-Refresh Trigger

4 Materialized View dibuat sebagai cache agregasi untuk dashboard. Setiap perubahan data di tabel sumber secara otomatis men-trigger REFRESH MV tanpa perlu manual.

Materialized View	Sumber	Insight yang Disediakan
mv_genre_summary	dim_genres	Genre paling cuan, paling diminati, tren tahunan
mv_revenue_summary	dim_revenue	Genre booming per dekade, tren industri film
mv_director_performance	dim_directors	Sutradara paling profitable, durasi karir
mv_country_performance	dim_country	Negara dominan per genre, genre favorit per negara

Hasil Validasi MV

Functions	Query	Query History
Materialized Views (4)	1	<code>SELECT * FROM public.mv_country_performance</code>
mv_country_performance	Data Output	Messages Notifications
mv_director_performance	Showing rows: 1 to 28	Page No: 1 of 1
mv_genre_summary	primary_country text	total_films bigint
mv_revenue_summary	total_revenue numeric	total_profit numeric
Operators	avg_roi_pct numeric	avg_rating numeric
Procedures	most_common_genre text	top_grossing_film text
Sequences	1 United States of Ameri...	3102 280520928071 189414222081 207.90 6.09 Comedy Avatar
Tables (3)	2 United Kingdom	374 36541235127 25848003976 241.72 6.46 Drama Skyfall
	3 Germany	200 16669142023 9338645341 127.39 6.23 Drama Harry Potter and the Chamber of Se

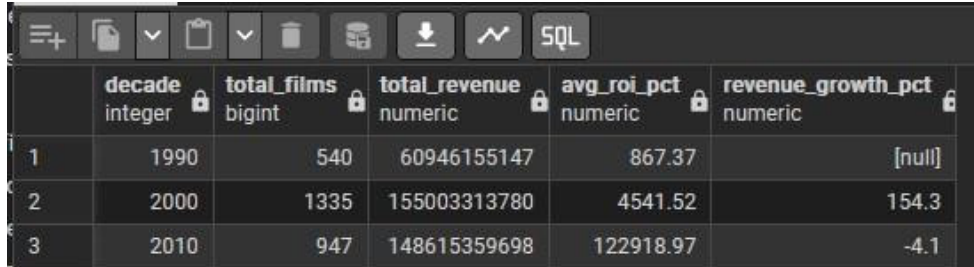
Query	Query History
1	<code>SELECT * FROM public.mv_revenue_summary</code>
Data Output	Messages Notifications
Showing rows: 1 to 119	Page No: 1 of 1
primary_genre text	decade integer
total_films bigint	mega_blockbuster bigint
blockbuster bigint	loss bigint
avg_roi_pct numeric	total_revenue numeric
total_profit numeric	
1 Action	2010 188 66 43 32 179.10 45854233587 30447602556
2 Adventure	2000 125 45 25 28 200.04 35464606380 24224115350
3 Action	2000 224 50 37 74 102.48 27386091171 15352691171
4 Comedy	2000 294 112 49 81 279.95 24723193779 16104347325
5 Drama	2000 322 102 43 118 401.28 22850141084 12930461632
6 Adventure	2010 61 17 20 10 173.35 19031318998 12291318998
7 Comedy	2010 175 71 31 34 229.22 16092294965 10495397279

Query	Query History
1	<code>SELECT * FROM public.mv_director_performance</code>
Data Output	Messages Notifications
Showing rows: 1 to 509	Page No: 1 of 1
director text	total_films bigint
avg_rating numeric	total_box_office numeric
total_profit numeric	roi_pct numeric
first_film_year integer	latest_film_year integer
career_years integer	
1 David Hand	3 0.70 535692636 531066213 11478.98 1937 1942 6
2 Alex Kendrick	4 0.68 78156869 76556869 4784.80 2003 2011 9
3 Victor Fleming	3 0.70 439294426 429890426 4571.36 1939 1944 6
4 Darren Lynn Bousman	5 0.62 456342667 432154541 1800.64 2005 2016 12
5 Guy Hamilton	6 0.65 507480445 478780445 1668.22 1964 1978 15
6 Henry Joost	3 0.61 432229120 402229120 1340.76 2011 2016 6

Query	Query History
1	<code>SELECT * FROM public.mv_genre_summary</code>
Data Output	Messages Notifications
Showing rows: 1 to 19	Page No: 1 of 1
genre text	total_films bigint
avg_rating numeric	avg_popularity numeric
total_revenue numeric	total_budget numeric
total_profit numeric	avg_runtime_minutes numeric
earliest_release integer	latest_release integer
top_grossing_film text	
1 Action	588 6.07 34.63 91459088189 34233272285 57225815904 111.0 1930 2016 Avatar
2 Adventure	288 6.39 44.60 70872316207 21705338117 49166978090 116.0 1939 2016 Captain
3 Drama	747 6.66 22.50 54101757502 19691630594 34410126908 121.0 1916 2016 Titanic
4 Comedy	634 6.06 20.32 53037118804 17399824524 35637294280 103.0 1933 2016 Shrek F
5 Animation	99 6.37 41.69 29595221289 8185432353 21409788936 91.0 1940 2016 Frozen
6 Fantasy	93 6.25 40.29 17057198167 5930438831 11126759336 108.0 1937 2016 Spider-
7 Science Ficti...	79 6.34 49.13 16176309703 4631307003 11545002700 113.0 1968 2016 The Av

3.5 CTE Analitik

CTE (Common Table Expressions) digunakan untuk membuat query lebih mudah dibaca, dengan 3 insight analitik siap pakai: top film per genre, sutradara paling konsisten, dan tren industri per dekade.



	decade integer	total_films bigint	total_revenue numeric	avg_roi_pct numeric	revenue_growth_pct numeric
1	1990	540	60946155147	867.37	[null]
2	2000	1335	155003313780	4541.52	154.3
3	2010	947	148615359698	122918.97	-4.1

Step 4 — PySpark ETL: Distributed Data Processing

PySpark digunakan untuk memproses data merge_raw dengan pendekatan distributed in-memory processing — berbeda dari pandas yang hanya bisa berjalan di satu mesin. Ini membuktikan kemampuan menangani data di skala yang pandas tidak mampu.

4.1 Perbandingan: pandas vs PySpark

Aspek	pandas	PySpark
Eksekusi	Eager — langsung diproses	Lazy — baru eksekusi saat .show()/write()
Batas data	Maks RAM laptop (~8-32 GB)	Petabyte (skala cluster)
SQL support	Tidak langsung	spark.sql() — full SQL
Cocok untuk	Eksplorasi, prototyping	Production pipeline, data besar

4.2 Alur ETL PySpark

EXTRACT → merge_raw dari PostgreSQL via JDBC (4.803 baris, 22 kolom)

TRANSFORM 1 — Ekstraksi kolom JSON:

genres → primary_genre

production_companies → primary_company

spoken_languages → primary_language

TRANSFORM 2 — Kalkulasi kolom baru:

profit = revenue - budget

roi_pct = (revenue - budget) / budget × 100

performance_label: Mega Blockbuster / Blockbuster / Profitable / Break Even / Loss

popularity_tier: Viral (>100) / Popular (>50) / Known (>20) / Niche

TRANSFORM 3 — Agregasi per genre (groupBy primary_genre):
 total_films, avg_rating, total_revenue, total_profit, avg_roi_pct
 mega_blockbuster_count, loss_count

LOAD 1 → public.pyspark_tmdb_finance (data per film, lengkap)
 LOAD 2 → public.pyspark_genre_aggregation (agregasi per genre, 20 rows)

4.3 Hasil Validasi

Tabel	Jumlah Rows	Keterangan
pyspark_tmdb_finance	~2.900	Film dengan budget > 0 DAN revenue > 0
pyspark_genre_aggregation	20	1 baris per genre (Action, Drama, Comedy, dll.)

The screenshot shows a database interface with a sidebar on the left listing tables like 'fact_yearly_partisi', 'merge_raw', 'pyspark_genre_aggregation', 'pyspark_tmdb_finance', 'pyspark_tmdb_result', 'tmdb_raw', 'Trigger Functions', 'Types', 'Views', and 'silver'. The main panel displays a query: `SELECT COUNT(*) FROM public.pyspark_tmdb_finance;`. Below the query, the 'Data Output' tab shows a single row with the column 'count' and the value '4803'.

The top screenshot shows a query: `SELECT performance_label, COUNT(*) as total FROM public.pyspark_tmdb_finance GROUP BY performance_label ORDER BY total DESC;`. The 'Data Output' tab shows a table with two columns: 'performance_label' (text) and 'total' (bigint). The results are:

performance_label	total
Mega Blockbuster	2313
Loss	1326
Profitable	629
Blockbuster	533
Break Even	2

The bottom screenshot shows a query: `SELECT primary_genre, total_films, total_revenue, avg_roi_pct FROM public.pyspark_genre_aggregation ORDER BY total_revenue DESC;`. The 'Data Output' tab shows a table with four columns: 'primary_genre' (text), 'total_films' (bigint), 'total_revenue' (bigint), and 'avg_roi_pct' (double precision). The results are:

primary_genre	total_films	total_revenue	avg_roi_pct
Action	588	91459088189	187.51
Adventure	288	70872316207	471.22
Drama	747	54101757502	1138398.15
Comedy	634	53037118804	425.95
Animation	99	29595221289	620.67
Fantasy	93	17057198167	370.35
Science Fiction	79	16176309703	418.92

Step 6 — Apache Kafka + Debezium CDC: Real-Time Streaming

Step 6 membangun sistem Change Data Capture (CDC) yang menangkap setiap perubahan data di PostgreSQL secara real-time dan mengalirkannya ke Kafka. Ini adalah pola arsitektur yang digunakan di skala besar seperti Uber (130 juta event/detik).

Membuka Cluster di Kafka : <http://localhost:9000>

6.1 Komponen Stack Kafka

Container	Image	Port	Fungsi
tmdb_zookeeper	confluentinc/cp-zookeeper:7.4.0	2181	Koordinasi Kafka cluster
tmdb_kafka	confluentinc/cp-kafka:7.4.0	29092:29092	Message broker, penyimpan event stream
tmdb_debezium	debezium/connect:2.4	8083:8083	CDC connector PostgreSQL → Kafka
tmdb_kafka_man	hlebalbau/kafkamanager	9000:9000	Web UI monitoring Kafka

6.2 Konfigurasi WAL PostgreSQL

Untuk Debezium bisa menangkap perubahan, PostgreSQL harus diset ke mode WAL logical:

```
# Masuk ke container PostgreSQL
docker exec -it tmdb_postgres bash

# Set WAL level ke logical
echo "wal_level = logical" >> /var/lib/postgresql/data/postgresql.conf

# Restart container
docker restart tmdb_postgres

-- Verifikasi di pgAdmin:
SHOW wal_level; -- harus menampilkan: logical

-- Cek publikasi Debezium:
SELECT * FROM pg_publication;
-- pubname: dbz_publication | puballtables: true
-- pubinsert/pubupdate/pubdelete: true
```

6.3 Hasil: 16 Kafka Topics

Debezium secara otomatis membuat 1 topic per tabel yang di-capture:

Topic Kafka	Sumber Tabel
-------------	--------------

tmdb.silver_mart.dim_actor	silver_mart.dim_actor
tmdb.silver_mart.dim_company	silver_mart.dim_company
tmdb.silver_mart.dim_country	silver_mart.dim_country
tmdb.silver_mart.dim_directors	silver_mart.dim_directors
tmdb.silver_mart.dim_genres	silver_mart.dim_genres
tmdb.silver_mart.dim_language	silver_mart.dim_language
tmdb.silver_mart.dim_movies	silver_mart.dim_movies
tmdb.silver_mart.dim_revenue	silver_mart.dim_revenue
tmdb.silver_mart.dim_yearlysum	silver_mart.dim_yearlysum
tmdb.silver_mart.fact_vs	silver_mart.fact_vs
tmdb.silver_mart.fact_yearly	silver_mart.fact_yearly
+ 5 internal Kafka topics	__consumer_offsets, connect_configs, dll.

Topic	# Partitions	# Brokers	Brokers Spread %	Brokers Skew %	Brokers Leader Skew %	# Replicas	Under Replicated %
__consumer_offsets	50	1	100	0	0	1	0
connect_configs	1	1	100	0	0	1	0
connect_offsets	25	1	100	0	0	1	0
connect_statuses	5	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_actors	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_company	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_country	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_directors	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_genres	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_language	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_movies	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_revenue	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.dim_yearlysum	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.fact_vs	1	1	100	0	0	1	0
tmdb.silver_mart_silver_mart.fact_yearly	1	1	100	0	0	1	0

Step 7 — ClickHouse OLAP: Column-Oriented Analytics

ClickHouse adalah database OLAP column-oriented yang jauh lebih cepat untuk analitik agregasi dibanding PostgreSQL (row-oriented). Di sinilah Medallion Architecture diimplementasikan ulang dalam ClickHouse: Bronze (Kafka Engine) → Silver (ReplacingMergeTree) → Gold (View).

Cek Clickhouse di : <http://localhost:18123/play>

7.1 Perbedaan PostgreSQL vs ClickHouse

Aspek	PostgreSQL (OLTP)	ClickHouse (OLAP)
Cara simpan	Per baris (row-oriented)	Per kolom (column-oriented)
Kecepatan analitik	Lebih lambat untuk agregasi	Jauh lebih cepat untuk analisis kolom
Use case	Transaksi: order, banking	Analitik: BI, Spotify Wrapped
MV auto-update	Tidak — perlu trigger manual	Ya — otomatis tanpa trigger

7.2 Arsitektur 3 Layer di ClickHouse

BRONZE: Kafka Engine Table (dim_movies_queue) — Langsung consume dari Kafka topic via ENGINE = Kafka.

SILVER: ReplacingMergeTree Table (dim_movies) — Simpan data final, hindari duplikat via ENGINE = ReplacingMergeTree(insert_time).

GOLD: View (tmdb_movies_gold) — Tambah kolom kalkulasi: profit, performance_label, popularity_tier. WHERE budget > 0 AND revenue > 0.

7.3 Objek di ClickHouse (9 Objek Total)

Nama Objek	Tipe	Engine / Keterangan
dim_movies	Base Table	ReplacingMergeTree — data final silver layer
dim_movies_queue	Kafka Queue	Kafka Engine — consume dari topic
dim_movies_mv	Materialized View	Jembatan queue → base table (otomatis)
dim_revenue	Base Table	ReplacingMergeTree
dim_revenue_queue	Kafka Queue	Kafka Engine
dim_revenue_mv	Materialized View	Otomatis
fact_yearly	Base Table	ReplacingMergeTree
fact_yearly_queue	Kafka Queue	Kafka Engine

fact_yearly_mv	Materialized View	Otomatis
----------------	-------------------	----------

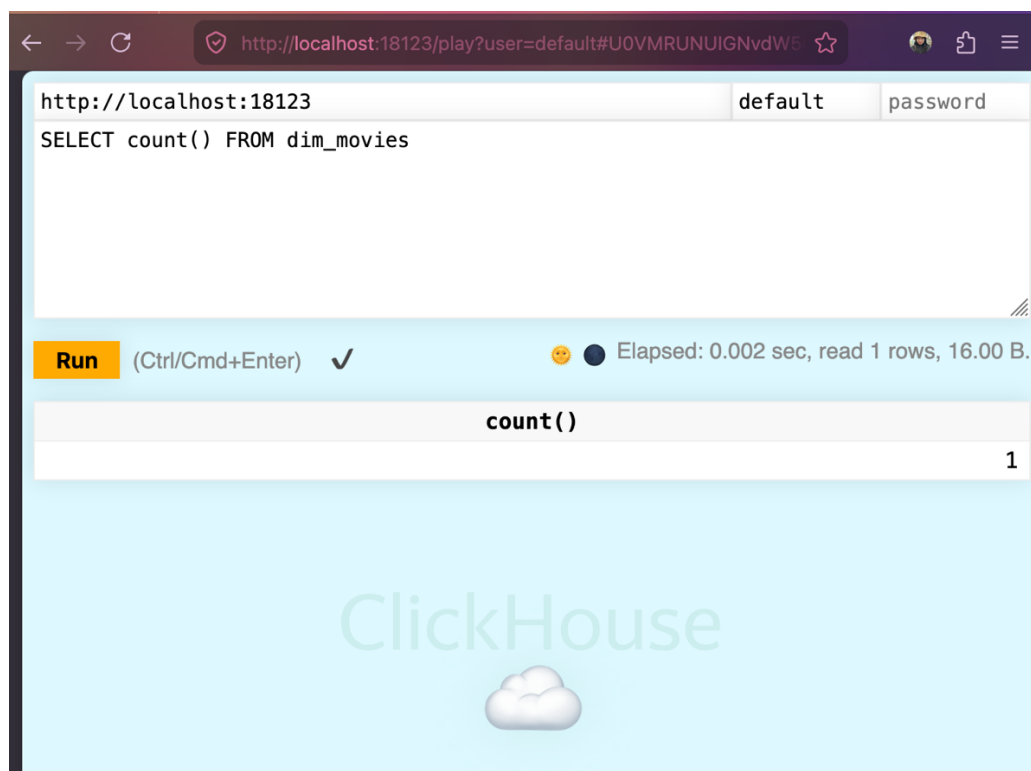
```

● macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project % curl http://localhost:18123/ --data-binary "SHOW TABLES"
dim_movies
dim_movies_mv
dim_movies_queue
dim_revenue
dim_revenue_mv
dim_revenue_queue
fact_yearly
fact_yearly_mv
fact_yearly_queue
○ macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project % 
Ln 54, Col 34 (1512 selected)  Spaces: 4  UTF-8  LF  {}  MS SQL

```

7.4 Hasil Validasi Data di ClickHouse

Tabel	Jumlah Rows	Keterangan
dim_movies	4.802 rows	Data mengalir dari Kafka CDC
dim_revenue	3.229 rows	Film dengan data keuangan lengkap
fact_yearly	363 rows	Agregasi per tahun per genre



Step 8 — MinIO Object Storage: Modern HDFS Replacement

MinIO adalah object storage modern yang kompatibel dengan Amazon S3, digunakan untuk menyimpan hasil mart layer dalam format Parquet.

Masuk ke MINIO = <http://localhost:9001/browser> loginnya = minioadmin/minioadmin

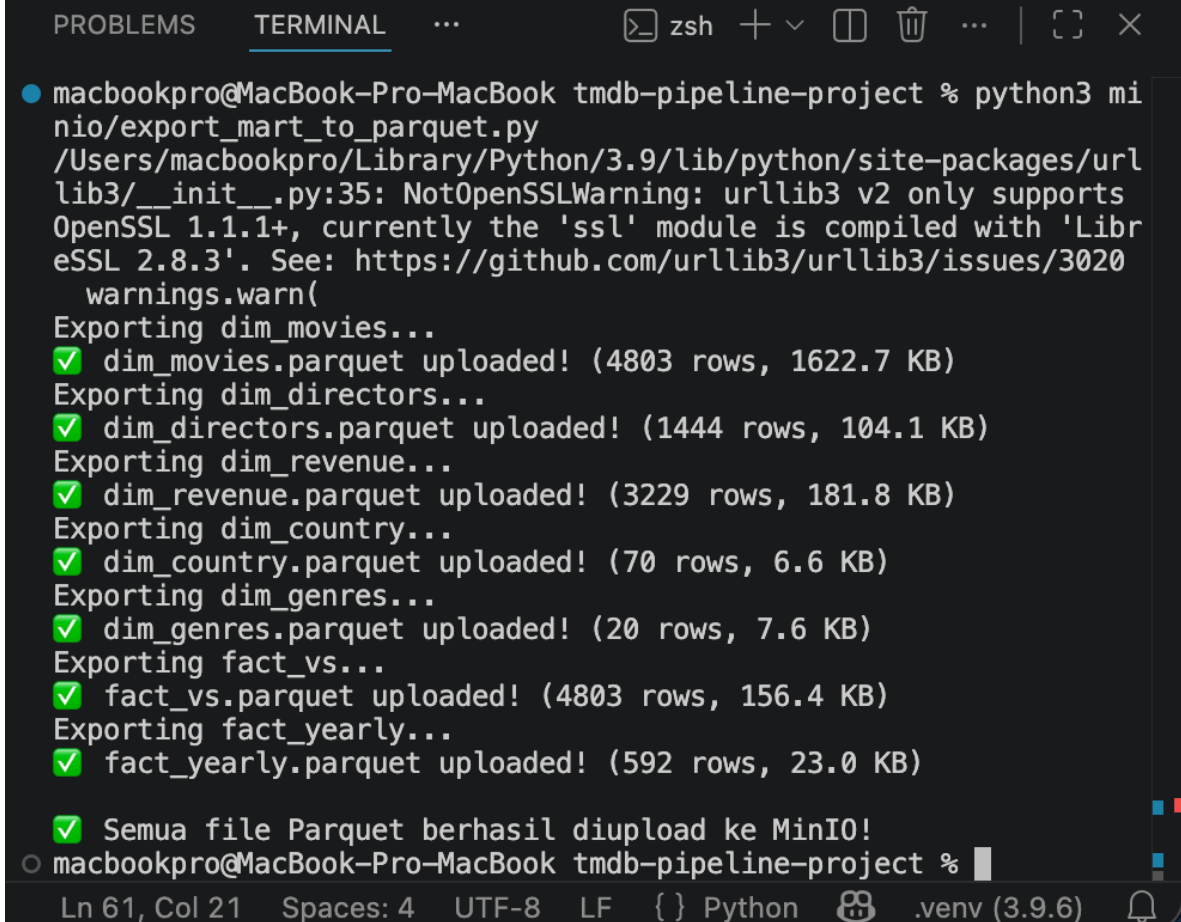
8.1 Konfigurasi

```
docker run -d -p 9002:9000 -p 9001:9001 --name tmdb_minio \  
-v C:\data\tmdb-minio\data \  
-e MINIO_ROOT_USER=minioadmin \  
-e MINIO_ROOT_PASSWORD=minioadmin \  
quay.io/minio/minio server /data --console-address ':9001'
```

Akses Web UI : <http://localhost:9001>

Username : minioadmin

Password : minioadmin

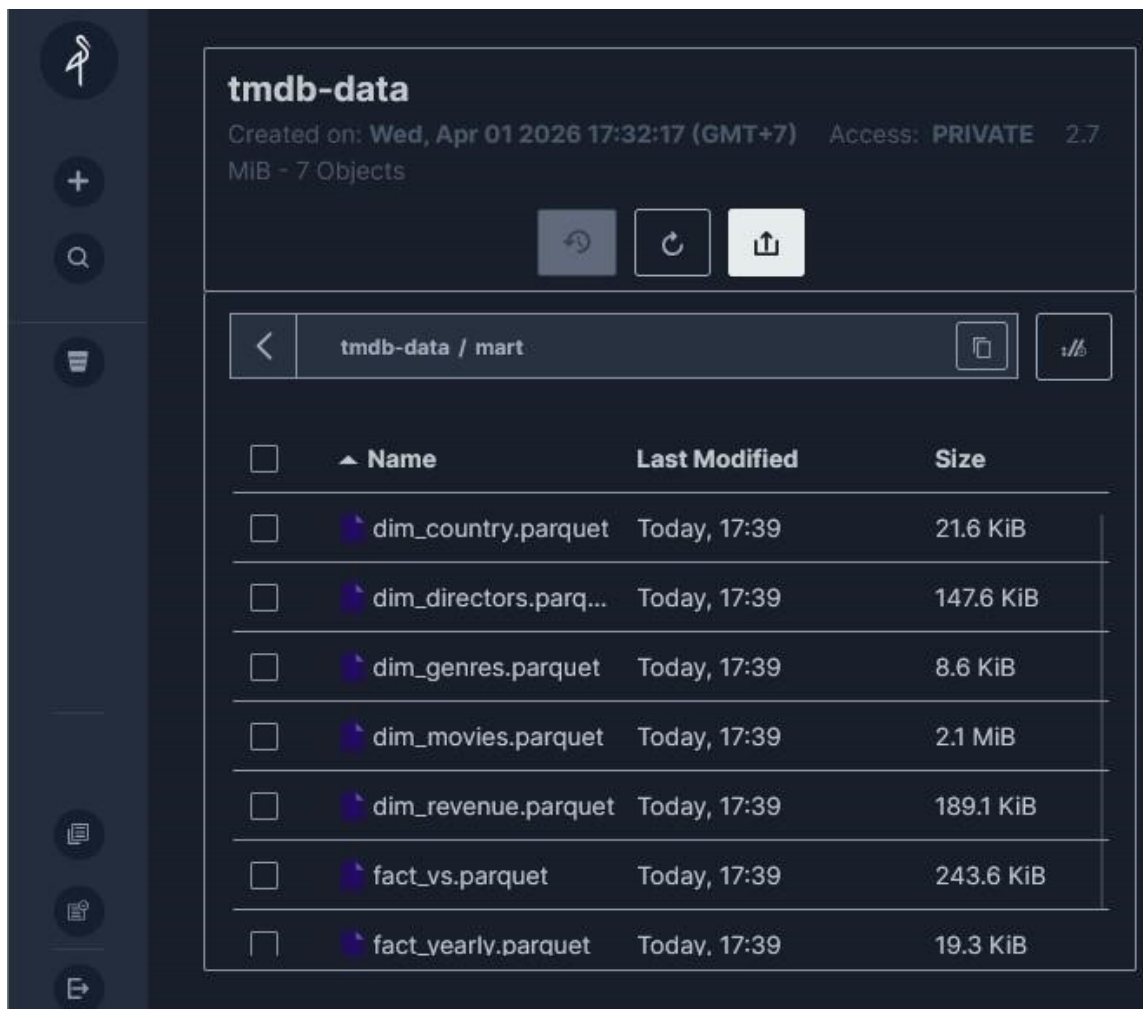


```
PROBLEMS  TERMINAL  ...  zsh  +  -  [ ]  [ ]  ...  |  [ ]  X  
● macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project % python3 mi  
nio/export_mart_to_parquet.py  
/Users/macbookpro/Library/Python/3.9/lib/python/site-packages/url  
lib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports  
OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'Libr  
eSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020  
  warnings.warn(  
Exporting dim_movies...  
✓ dim_movies.parquet uploaded! (4803 rows, 1622.7 KB)  
Exporting dim_directors...  
✓ dim_directors.parquet uploaded! (1444 rows, 104.1 KB)  
Exporting dim_revenue...  
✓ dim_revenue.parquet uploaded! (3229 rows, 181.8 KB)  
Exporting dim_country...  
✓ dim_country.parquet uploaded! (70 rows, 6.6 KB)  
Exporting dim_genres...  
✓ dim_genres.parquet uploaded! (20 rows, 7.6 KB)  
Exporting fact_vs...  
✓ fact_vs.parquet uploaded! (4803 rows, 156.4 KB)  
Exporting fact_yearly...  
✓ fact_yearly.parquet uploaded! (592 rows, 23.0 KB)  
  
✓ Semua file Parquet berhasil diupload ke MinIO!  
○ macbookpro@MacBook-Pro-MacBook tmdb-pipeline-project %  
Ln 61, Col 21  Spaces: 4  UTF-8  LF  {} Python  [ ]  .venv (3.9.6)
```

8.2 Hasil Upload — Bucket: tmdb-data/mart/

Total: 7 file Parquet, 2.7 MiB

File Parquet	Jumlah Rows	Ukuran	Tanggal Upload
dim_movies.parquet	4.802	2.1 MiB	01 Apr 2026, 17:39
dim_directors.parquet	2.349	147.6 KiB	01 Apr 2026, 17:39
fact_vs.parquet	4.867	243.6 KiB	01 Apr 2026, 17:39
dim_revenue.parquet	3.229	189.1 KiB	01 Apr 2026, 17:39
dim_country.parquet	70	21.6 KiB	01 Apr 2026, 17:39
fact_yearly.parquet	363	19.3 KiB	01 Apr 2026, 17:39
dim_genres.parquet	19	8.6 KiB	01 Apr 2026, 17:39



Step 9 — Streamlit Dashboard: Visualisasi Data

Dashboard interaktif dengan Streamlit (Python library) yang terhubung langsung ke data di PostgreSQL. Dashboard menampilkan scorecard KPI, chart, dan tabel dengan filter interaktif.

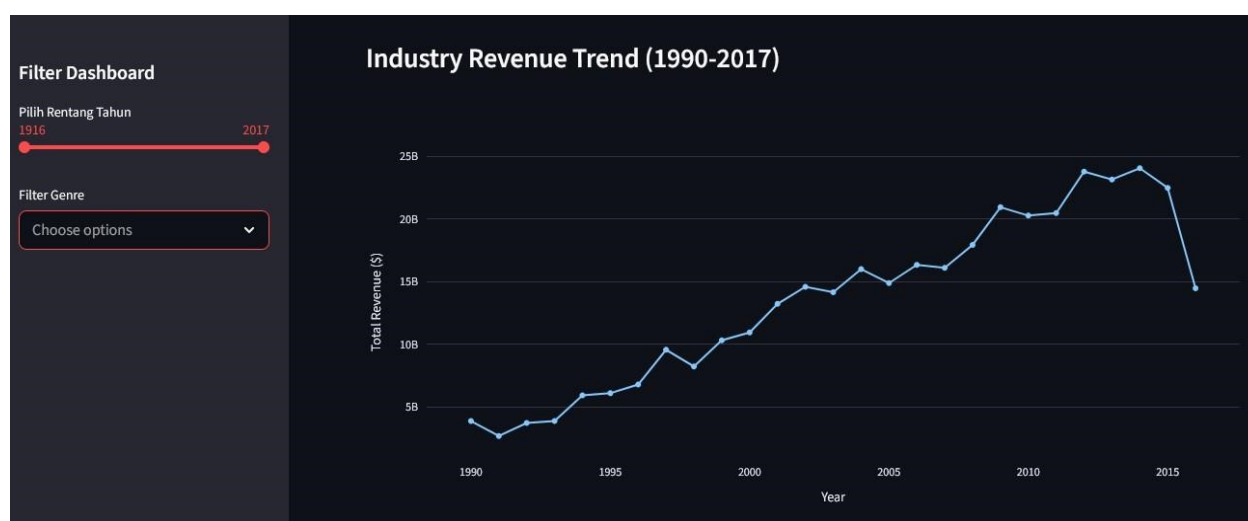
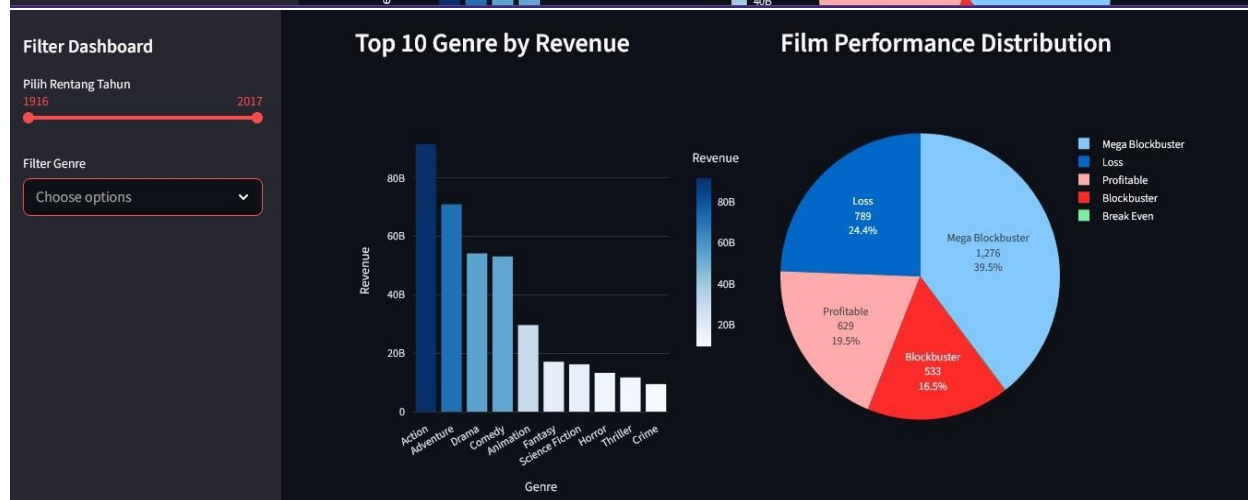
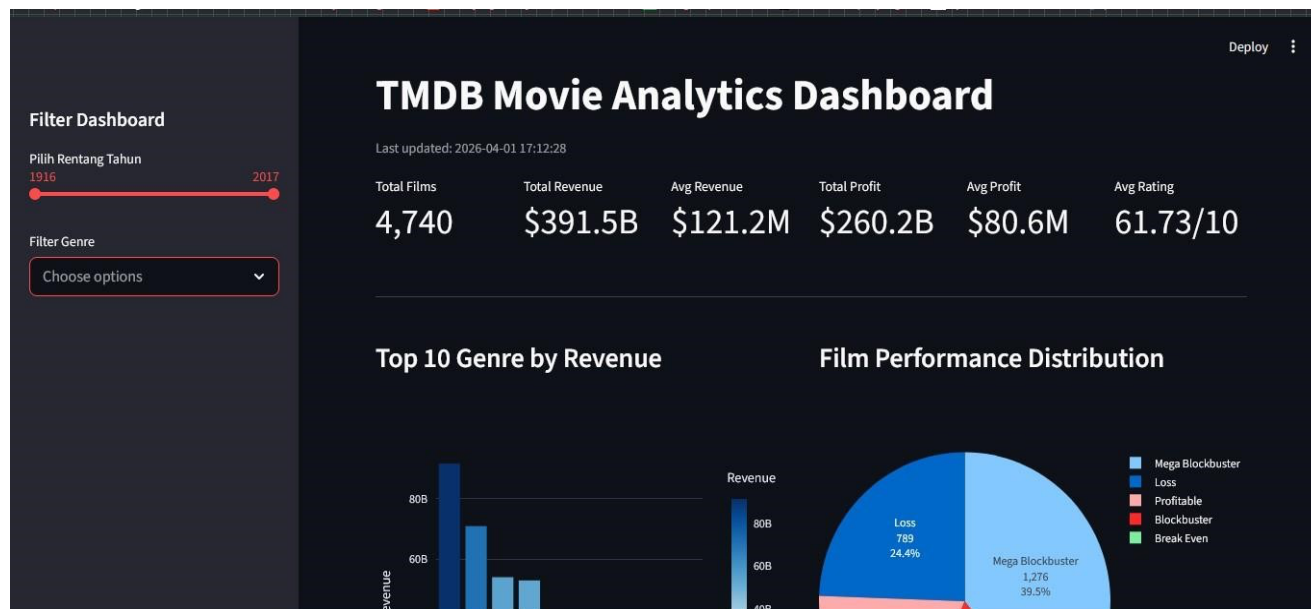
Cek dashboard Streamlit = <http://localhost:8501/>

9.1 Fitur Dashboard

Fitur	Keterangan
Filter Rentang Tahun	Slider untuk memilih tahun rilis (misal: 1959–2004)
Filter Genre	Multi-select untuk memfilter genre (Comedy, Adventure, dll.)
Scorecard KPI	Total Films, Total Revenue, Avg Revenue, Total Profit, Avg Profit, Avg Rating
Top 10 Genre by Revenue	Bar chart revenue per genre
Film Performance Distribution	Pie chart distribusi Mega Blockbuster / Blockbuster / Profitable / Loss
Top 20 Highest Revenue Films	Tabel detail dengan kolom Title, Genre, Lead Actor, Director, Revenue, Performance

9.2 Contoh Data Dashboard

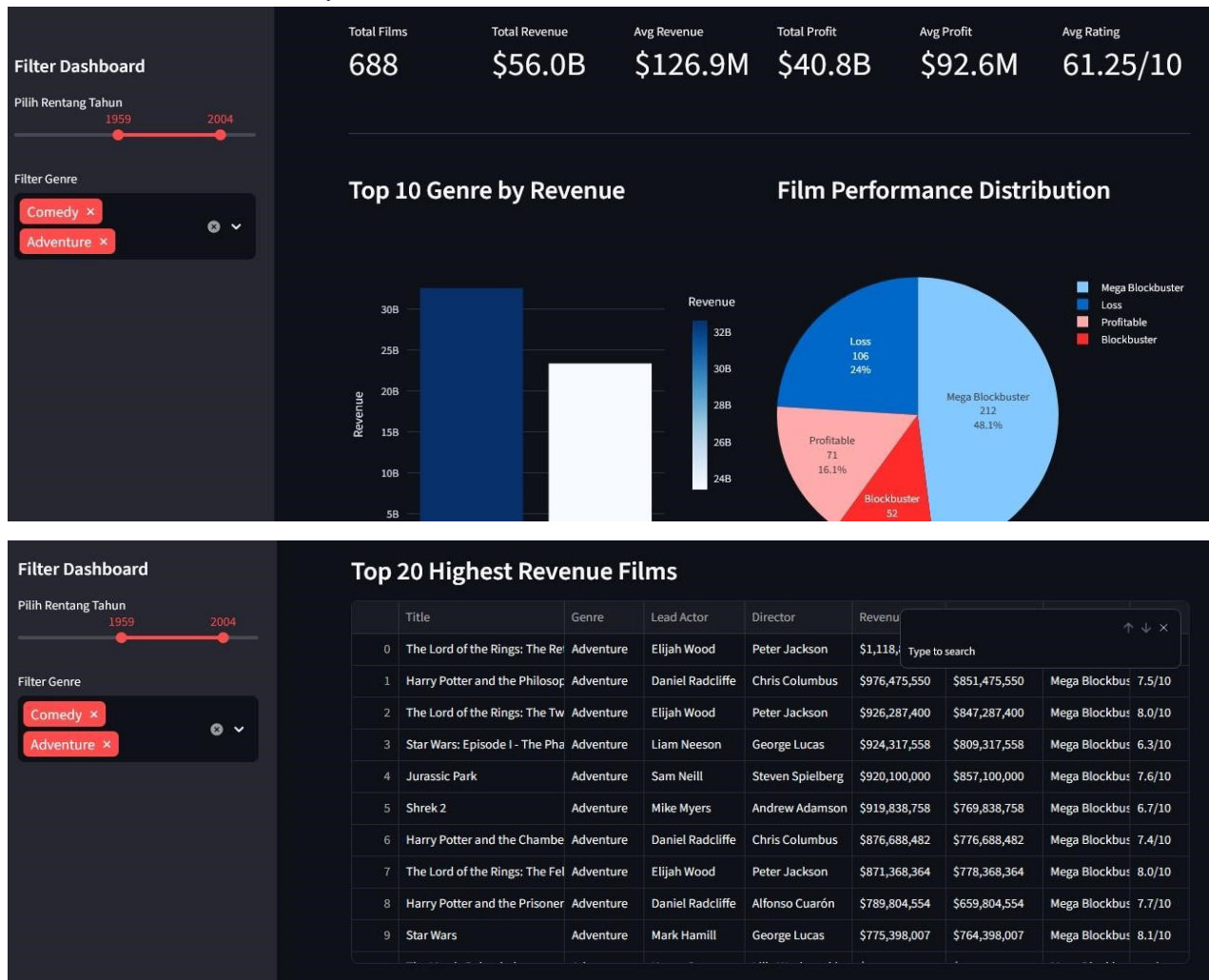
Filter: 1959–2004, Genre: Comedy + Adventure — Film tertinggi dalam filter ini: The Lord of the Rings: The Return of the King (\$1.118B revenue, Adventure, sutradara Peter Jackson)





9.2 Contoh Data Dashboard (Filter: 1959–2004, Genre: Comedy + Adventure)

Film tertinggi dalam filter ini: The Lord of the Rings: The Return of the King (\$1.118B revenue, Adventure, Peter Jackson)



Step 10 — Streamlit Dashboard: Visualisasi Data

Apache Airflow digunakan untuk mengotomasi dan menjadwalkan jalannya pipeline TMDB secara end-to-end. DAG (Directed Acyclic Graph) bernama `tmdb_pipeline_dag` mengatur urutan eksekusi 4 task secara berurutan dan terjadwal otomatis.

Membuka Airflow = <http://localhost:8082>

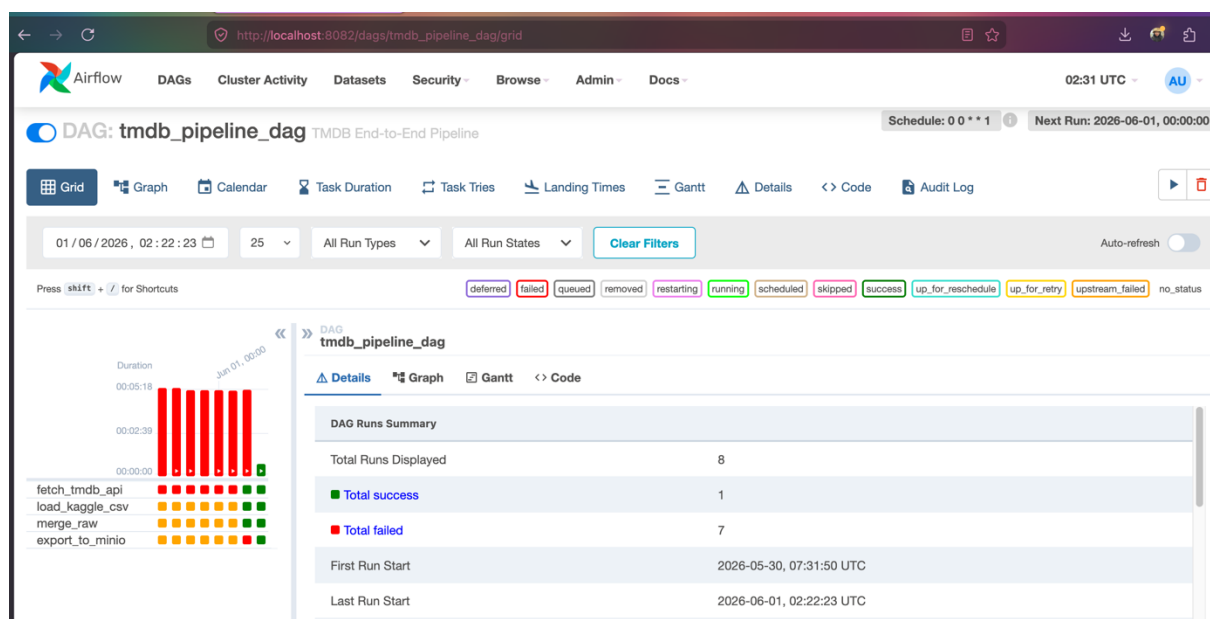
10.1 Konfigurasi DAG

Komponen	Nilai
DAG ID	<code>tmdb_pipeline_dag</code>
Deskripsi	TMDB End-to-End Pipeline
Schedule	<code>0 0 * * 1</code> (setiap Senin tengah malam)
Next Run	2026-06-01, 00:00:00
First Run Start	2026-05-30, 07:31:50 UTC
Last Run Start	2026-06-01, 02:22:23 UTC

10.2 Urutan Task dalam DAG

Urutan	Task	Fungsi
1	<code>fetch_tmdb_api</code>	Ambil data film terbaru dari TMDb API live
2	<code>load_kaggle_csv</code>	Load file CSV Kaggle ke PostgreSQL
3	<code>merge_raw</code>	Gabungkan kedua sumber menjadi tabel <code>merge_raw</code>
4	<code>export_to_minio</code>	Export hasil mart ke MinIO dalam format Parquet

10.3 Hasil Eksekusi DAG



Metrik	Nilai
Total Runs Displayed	8
Total Success	1
Total Failed	7

Catatan: Kegagalan run terjadi selama fase development dan konfigurasi environment. Run terakhir berhasil sukses setelah semua dependency container (PostgreSQL, MinIO) dalam kondisi aktif dan koneksi dikonfigurasi dengan benar.

Lampiran: Ringkasan Semua Objek Database

1) PostgreSQL — tmdb_db

Schema	Tabel / Objek	Tipe	Keterangan
public	merge_raw	Tabel	Bronze: 4.803 rows, 22 kolom, hasil merge 2 CSV
public	tmdb_raw	Tabel	Bronze: 80 rows dari TMDb API
public	fact_yearly_partisi	Tabel Induk	Partisi RANGE berdasarkan release_year
public	fact_yearly_1920s ... 2010s	Partisi (10x)	Sub-tabel per dekade
public	pyspark_tmdb_finance	Tabel	Hasil PySpark ETL — per film dengan kalkulasi
public	pyspark_genre_aggregation	Tabel	Hasil PySpark ETL — agregasi per genre (20 rows)
public	pyspark_tmdb_result	Tabel	Hasil PySpark ETL awal (versi sederhana)
public	nifi_tmdb_movie	Tabel	Hasil Apache NiFi pipeline dari CSV
silver_mart	dim_movies	Tabel dbt	Master film lengkap — 4.802 rows
silver_mart	dim_genres	Tabel dbt	Ringkasan per genre — 19 rows
silver_mart	dim_directors	Tabel dbt	Profil sutradara — 2.349 rows
silver_mart	dim_actor	Tabel dbt	Profil aktor
silver_mart	dim_country	Tabel dbt	Ringkasan per negara — 70 rows
silver_mart	dim_language	Tabel dbt	Ringkasan per bahasa
silver_mart	dim_company	Tabel dbt	Profil rumah produksi
silver_mart	dim_revenue	Tabel dbt	Fakta keuangan — 3.229 rows
silver_mart	dim_yearlysum	Tabel dbt	Ringkasan tahunan
silver_mart	fact_vs	Tabel dbt	Rekonsiliasi 2 sumber — 4.867 rows
silver_mart	fact_yearly	Tabel dbt	Tren per tahun per genre — 363 rows
public	mv_genre_summary	MV	Cache agregasi genre
public	mv_revenue_summary	MV	Cache analisis keuangan per dekade
public	mv_director_performance	MV	Cache profil sutradara
public	mv_country_performance	MV	Cache profil negara

2) ClickHouse — 9 Objek

Nama	Tipe	Engine	Rows
dim_movies	Base Table	ReplacingMergeTree	9.604
dim_movies_queue	Kafka Table	Kafka Engine	—
dim_movies_mv	Materialized View	—	—

dim_revenue	Base Table	ReplacingMergeTree	6.458
dim_revenue_queue	Kafka Table	Kafka Engine	—
dim_revenue_mv	Materialized View	—	—
fact_yearly	Base Table	ReplacingMergeTree	726
fact_yearly_queue	Kafka Table	Kafka Engine	—
fact_yearly_mv	Materialized View	—	—

```
SHOW TABLES

Query id: 0c75cca0-9a87-4b07-bcf1-d782f275b885

+----+-----+
| name |
+----+-----+
1.  dim_movies
2.  dim_movies_mv
3.  dim_movies_queue
4.  dim_revenue
5.  dim_revenue_mv
6.  dim_revenue_queue
7.  fact_yearly
8.  fact_yearly_mv
9.  fact_yearly_queue
+----+-----+

9 rows in set. Elapsed: 0.021 sec.
```

3) Container Docker yang Berjalan

Service	Container Name	Image	Port
Airflow Scheduler	tmdb_airflow_scheduler	apache/airflow:2.8.1	-
Airflow Webserver	tmdb_airflow_web	apache/airflow:2.8.1	8082:8080
Airflow Init	tmdb_airflow_init	apache/airflow:2.8.1	-
Debezium Connect	tmdb_debezium	debezium/connect:2.4	8083:8083
pgAdmin	tmdb_pgadmin	dpage/pgadmin4	5050:80
Kafka Broker	tmdb_kafka	confluentinc/cp-kafka:7.4.0	29092:29092
Kafka Manager	tmdb_kafka_man	hlebalbau/kafka-manager:latest	9000:9000
PostgreSQL	tmdb_postgres	postgres:13	5432:5432
ClickHouse	tmdb_clickhouse	clickhouse/clickhouse-server:23.8	18123:8123
ZooKeeper	tmdb_zookeeper	confluentinc/cp-zookeeper:7.4.0	-
MinIO	tmdb_minio	minio/minio:latest	9002:9000